

# Data Analysis Report Using Python (Pandas & Matplotlib)

## 1. Load the Dataset

```
import pandas as pd

df = pd.read_csv("Superstore.csv")

print("Dataset Loaded Successfully")
print(df)
```

### Output:

Dataset Loaded Successfully

|      | Row ID | ... | Sales    |
|------|--------|-----|----------|
| 0    | 1      | ... | 261.9600 |
| 1    | 2      | ... | 731.9400 |
| 2    | 3      | ... | 14.6200  |
| 3    | 4      | ... | 957.5775 |
| 4    | 5      | ... | 22.3680  |
| ...  | ...    | ... | ...      |
| 9795 | 9796   | ... | 3.7980   |
| 9796 | 9797   | ... | 10.3680  |
| 9797 | 9798   | ... | 235.1880 |
| 9798 | 9799   | ... | 26.3760  |
| 9799 | 9800   | ... | 10.3840  |

[9800 rows x 18 columns]

## 2. Check Shape

```
print("Dataset Shape:")

print(df.shape)
```

### Output:

Dataset Shape:  
(9800, 18)

## 3. Check Columns

```
print("Columns in Dataset:")

print(df.columns)
```

### Output:

Columns in Dataset:

```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
      'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',
      'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
      'Product Name', 'Sales'],
      dtype='object')
```

#### 4. Check Missing Values

```
print("Missing Values:")  
  
print(df.isnull().sum())
```

##### Output:

```
Missing Values:  
Row ID      0  
Order ID    0  
Order Date  0  
Ship Date   0  
Ship Mode   0  
Customer ID 0  
Customer Name 0  
Segment     0  
Country     0  
City        0  
State       0  
Postal Code 11  
Region      0  
Product ID  0  
Category    0  
Sub-Category 0  
Product Name 0  
Sales       0  
dtype: int64
```

#### 5. Check Duplicates

```
print("Total Duplicate Rows:")  
  
print(df.duplicated().sum())
```

##### Output:

```
Total Duplicate Rows:  
0
```

#### 6. Remove Duplicates

```
df = df.drop_duplicates()  
  
print("Duplicates Removed")  
  
print("Remaining Duplicates:")  
  
print(df.duplicated().sum())
```

##### Output:

```
Duplicates Removed  
Remaining Duplicates:  
0
```

## 7. Detect Outliers

```
import numpy as np

numeric_cols = df.select_dtypes(
    include=np.number
).columns

for col in numeric_cols:

    Q1 = df[col].quantile(0.25)

    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR

    upper = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower) |
        (df[col] > upper)
    ]

    print(col, "Outliers:", len(outliers))
```

### Output:

```
Row ID Outliers: 0
Postal Code Outliers: 0
Sales Outliers: 1145
```

## 8. Generate Descriptive Statistics

```
print(df.describe())
```

### Output:

```
Row ID Outliers: 0
```

|       | Row ID      | Postal Code  | Sales       |
|-------|-------------|--------------|-------------|
| count | 9800.000000 | 9789.000000  | 9800.000000 |
| mean  | 4900.500000 | 55273.322403 | 230.769059  |
| std   | 2829.160653 | 32041.223413 | 626.651875  |
| min   | 1.000000    | 1040.000000  | 0.444000    |
| 25%   | 2450.750000 | 23223.000000 | 17.248000   |
| 50%   | 4900.500000 | 58103.000000 | 54.490000   |
| 75%   | 7350.250000 | 90008.000000 | 210.605000  |

```
max 9800.000000 99301.000000 22638.480000
```

Postal Code Outliers: 0

|       | Row ID      | Postal Code  | Sales        |
|-------|-------------|--------------|--------------|
| count | 9800.000000 | 9789.000000  | 9800.000000  |
| mean  | 4900.500000 | 55273.322403 | 230.769059   |
| std   | 2829.160653 | 32041.223413 | 626.651875   |
| min   | 1.000000    | 1040.000000  | 0.444000     |
| 25%   | 2450.750000 | 23223.000000 | 17.248000    |
| 50%   | 4900.500000 | 58103.000000 | 54.490000    |
| 75%   | 7350.250000 | 90008.000000 | 210.605000   |
| max   | 9800.000000 | 99301.000000 | 22638.480000 |

Sales Outliers: 1145

|       | Row ID      | Postal Code  | Sales        |
|-------|-------------|--------------|--------------|
| count | 9800.000000 | 9789.000000  | 9800.000000  |
| mean  | 4900.500000 | 55273.322403 | 230.769059   |
| std   | 2829.160653 | 32041.223413 | 626.651875   |
| min   | 1.000000    | 1040.000000  | 0.444000     |
| 25%   | 2450.750000 | 23223.000000 | 17.248000    |
| 50%   | 4900.500000 | 58103.000000 | 54.490000    |
| 75%   | 7350.250000 | 90008.000000 | 210.605000   |
| max   | 9800.000000 | 99301.000000 | 22638.480000 |

## 9. Perform GroupBy Analysis

# Sales by Category

```
print("Sales by Category")
```

```
print(  
    df.groupby("Category")["Sales"].sum()  
)
```

**Output:**

Sales by Category

Category

Furniture 728658.5757

Office Supplies 705422.3340

Technology 827455.8730

Name: Sales, dtype: float64

# Sales by Region

```
print("\nSales by Region")
```

```
print(  
    df.groupby("Region")["Sales"].sum()  
)
```

**Output:**

Sales by Region

Region

Central 492646.9132

```
East    669518.7260
South   389151.4590
West    710219.6845
Name: Sales, dtype: float64
```

```
# Average Sales by Segment
```

```
print("\nAverage Sales by Segment")
```

```
print(
    df.groupby("Segment")["Sales"].mean()
)
```

### Output:

```
Average Sales by Segment
Segment
Consumer    225.065777
Corporate    233.150720
Home Office  243.403309
Name: Sales, dtype: float64
```

## 10. Create 5 Charts

### Chart 1: Sales by Category

```
import matplotlib.pyplot as plt
```

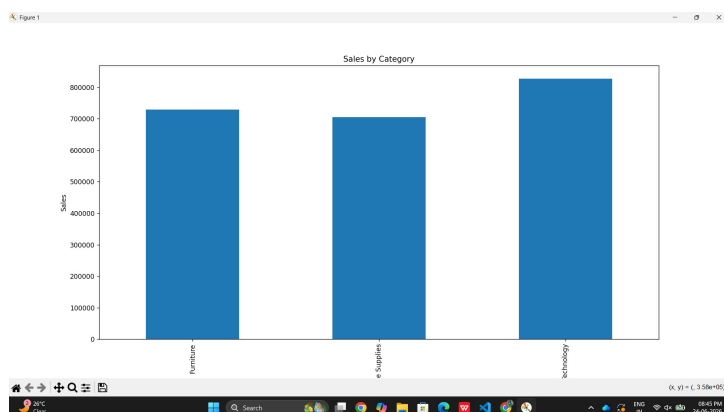
```
df.groupby("Category")["Sales"].sum().plot(
    kind="bar"
)
```

```
plt.title("Sales by Category")
```

```
plt.xlabel("Category")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```



## Chart 2: Sales by Region

```
df.groupby("Region")["Sales"].sum().plot(
    kind="bar"
)

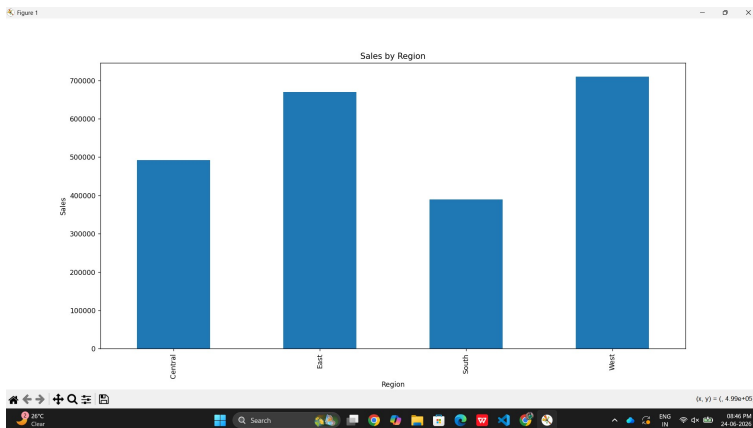
plt.title("Sales by Region")

plt.xlabel("Region")

plt.ylabel("Sales")

plt.show()
```

### Output:



## Chart 3: Category Distribution

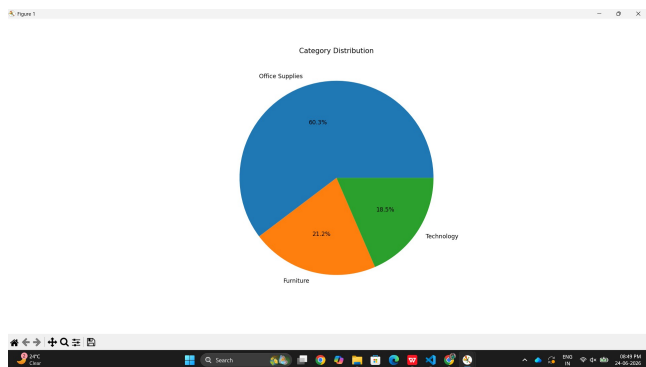
```
df["Category"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%"
)

plt.title("Category Distribution")

plt.ylabel("")

plt.show()
```

### Output:



**Chart 4: Sales Histogram**

```
plt.hist(
    df["Sales"],
    bins=20
)

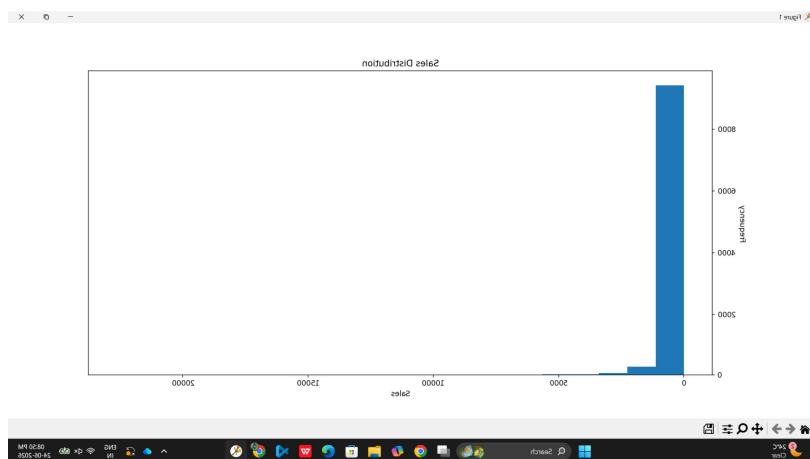
plt.title("Sales Distribution")

plt.xlabel("Sales")

plt.ylabel("Frequency")

plt.show()
```

**Output:**



**Chart 5: Scatter Plot**

```
plt.scatter(
    df["Row ID"],
    df["Sales"]
)

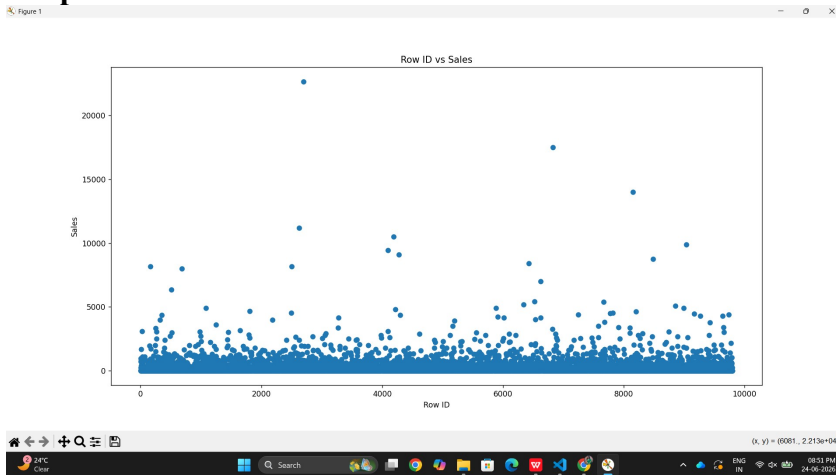
plt.title("Row ID vs Sales")

plt.xlabel("Row ID")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

### Output:



## 11. Find Correlations

```
corr = df.select_dtypes(  
    include=np.number  
).corr()
```

```
print("Correlation Matrix:")
```

```
print(corr)
```

### Output:

Correlation Matrix:

|             | Row ID   | Postal Code | Sales     |
|-------------|----------|-------------|-----------|
| Row ID      | 1.000000 | 0.013645    | 0.001151  |
| Postal Code | 0.013645 | 1.000000    | -0.024067 |
| Sales       | 0.001151 | -0.024067   | 1.000000  |

### Correlation Heatmap

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,6))
```

```
plt.imshow(  
    corr,  
    cmap="coolwarm"  
)
```

```
plt.colorbar()
```

```
plt.xticks(  
    range(len(corr.columns)),  
    corr.columns,  
    rotation=90  
)
```

```
plt.yticks(  
    range(len(corr.columns)),  
    corr.columns  
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

## Output:

